

**FACULTATEA DE AUTOMATICĂ ȘI
CALCULATOARE**
Departamentul de Calculatoare

Procesarea Imaginilor

**Crearea Filmelor 3D Anaglife prin Analiză
Stereoscopică**

Studenti: Ghineț Ioana-Teodora
Murariu Georgiana
Anul: III, CTI
Grupa: 30434

Cluj-Napoca, 2025

Cuprins

1	Introducere și Context Teoretic	3
1.1	Cerința Proiectului	3
1.2	Contextul Procesării Stereoscopice	3
1.3	Imaginea Anaglifă	4
2	Structura Proiectului și Tehnologii Folosite	5
2.1	Structura Directoarelor	5
2.2	Tehnologii Folosite	6
2.3	Meniul Aplicației	6
3	Arhitectura Sistemului și Pipeline-ul de Procesare	8
3.1	Descriere Generală	8
3.2	Diagrama Pipeline-ului	9
3.3	Module Principale	9
3.3.1	ImageLoader	9
3.3.2	DisparityMap	10
3.3.3	AnaglyphGenerator	10
3.3.4	VideoGenerator	11
4	Modele Matematice și Implementare	12
4.1	Calculul Hărții de Disparitate prin SAD	12
4.2	Normalizarea Depth Map-ului	13
4.3	Generarea Anaglifului Simplu	13
4.4	Generarea Anaglifului cu Depth Map	13
5	Fluxul de Rulare al Aplicației	15
5.1	Pregătirea Datelor	15
5.2	Conversia Dataset-ului Brut	15
5.3	Procesarea unei Singure Perechi Stereo	16
5.4	Procesarea Secvenței Complete	16
5.5	Generarea Video-ului Final	16
6	Analiza Performanței și Rezultate	17

6.1	Dataset-uri Utilizate	17
6.2	Analiza Complexității	18
6.3	Rezultate Obținute	18
6.4	Limitări	18
7	Concluzii, Contribuții și Dezvoltări Ulterioare	20
7.1	Concluzii	20
7.2	Contribuțiile Membrilor Echipei	20
7.3	Dezvoltări Ulterioare	21
A	Anexe: Snippet-uri de Cod Esențiale	23
A.1	Calculul Hărții de Disparitate	23
A.2	Normalizarea Hărții de Disparitate	25
A.3	Generarea Anaglifului Simplu	26
A.4	Generarea Anaglifului cu Depth Map	26
A.5	Încărcarea unei Perechi Stereo	27
A.6	Generarea Video-ului Final	28
A.7	Meniul Principal al Aplicației	30

Capitolul 1

Introducere și Context Teoretic

1.1 Cerința Proiectului

Proiectul are ca scop crearea unor secvențe video 3D anaglife pornind de la perechi de imagini stereo. Pentru fiecare cadru există două imagini: una corespunzătoare perspectivei din stânga și una corespunzătoare perspectivei din dreapta. Aceste imagini sunt prelucrate pentru a genera atât hărți de disparitate, cât și cadre anaglife care pot fi apoi combinate într-un fișier video de tip `.avi`.

Aplicația primește ca intrare imagini grayscale pe 8 biți/pixel, în format BMP, organizate în două directoare separate: unul pentru imaginile din stânga și unul pentru imaginile din dreapta. Pentru fiecare pereche stereo, sistemul generează:

- o hartă de disparitate / depth map;
- un cadru anaglif simplu;
- opțional, un cadru anaglif în care deplasarea este influențată de harta de disparitate;
- o secvență video finală în format `anaglyph_video.avi`.

Obiectivul principal al proiectului este dublu. Pe de o parte, se urmărește estimarea diferenței de poziție dintre aceleași puncte observate în cele două imagini stereo. Pe de altă parte, se urmărește generarea unei imagini color speciale, de tip roșu-cyan, care poate crea iluzia de profunzime atunci când este vizualizată cu ochelari anaglifi.

1.2 Contextul Procesării Stereoscopice

Viziunea stereoscopică reprezintă o ramură importantă a procesării imaginilor și a viziunii computerizate. Aceasta încearcă să reproducă modul în care sistemul vizual uman percepe

adâncimea. Omul are doi ochi aflați la o anumită distanță unul față de celălalt, iar fiecare ochi observă scena dintr-o perspectivă ușor diferită. Creierul folosește diferențele dintre cele două imagini pentru a deduce poziția obiectelor în spațiu.

În mod similar, într-un sistem stereo artificial, două camere sunt plasate la o anumită distanță una de cealaltă, numită *baseline*. Un punct 3D din scenă se proiectează în poziții diferite în imaginea stângă și în imaginea dreaptă. Diferența dintre coordonatele orizontale ale aceluiași punct poartă numele de **disparitate**.

În cazul ideal, camerele sunt aliniate orizontal și imaginile sunt rectificate. Astfel, căutarea punctului corespondent se poate face pe aceeași linie, doar pe direcția orizontală. Această presupunere simplifică foarte mult calculul, deoarece nu mai este nevoie să se caute corespondentul într-o zonă bidimensională, ci doar de-a lungul unei linii.

Relația teoretică dintre adâncime și disparitate este:

$$Z = \frac{f \cdot B}{d} \quad (1.1)$$

unde:

- Z este distanța până la punctul observat;
- f este distanța focală a camerei;
- B este distanța dintre cele două camere;
- d este disparitatea.

Din această relație se observă că adâncimea este invers proporțională cu disparitatea. Obiectele mai apropiate au o disparitate mai mare, iar obiectele îndepărtate au o disparitate mai mică.

1.3 Imaginea Anaglifă

O imagine anaglifă este o imagine color obținută prin combinarea a două perspective stereo într-un singur cadru. În varianta roșu-cyan, informația pentru ochiul stâng este introdusă în canalul roșu, iar informația pentru ochiul drept este introdusă în canalele verde și albastru. Atunci când imaginea este privită prin ochelari anaglifi, fiecare ochi primește o informație diferită, iar creierul interpretează diferența ca efect de adâncime.

În proiectul de față, imaginile inițiale sunt grayscale. Din acest motiv, fiecare pixel are o singură valoare de intensitate. Pentru a crea imaginea anaglifă, această valoare este copiată în canalele de culoare corespunzătoare.

Capitolul 2

Structura Proiectului și Tehnologii Folosite

2.1 Structura Directoarelor

Proiectul este organizat modular, folosind fișiere separate pentru fiecare responsabilitate importantă. Structura generală este următoarea:

```
1 Project_PI/
2 |-- CMakeLists.txt
3 |-- README.md
4 |-- .gitignore
5 |-- src/
6 |   |-- main.cpp
7 |   |-- ImageLoader.cpp
8 |   |-- ImageLoader.h
9 |   |-- AnaglyphGenerator.cpp
10 |  |-- AnaglyphGenerator.h
11 |  |-- DisparityMap.cpp
12 |  |-- DisparityMap.h
13 |  |-- VideoGenerator.cpp
14 |  |-- VideoGenerator.h
15 |-- data/
16 |   |-- raw/
17 |   |-- left/
18 |   |-- right/
19 |   |-- output/
20 |       |-- frames/
21 |       |-- depth_maps/
22 |       |-- anaglyph_video.avi
23 |-- docs/
24 |   |-- screenshots/
```

Directorul `src` conține codul sursă al aplicației. Directorul `data` conține datele de intrare și rezultatele generate. În `data/left` se află imaginile pentru camera stângă, iar în `data/right` imaginile pentru camera dreaptă. Rezultatele se salvează în `data/output`.

2.2 Tehnologii Folosite

Proiectul a fost realizat folosind următoarele tehnologii:

- **C++17** – limbajul principal de implementare;
- **OpenCV** – bibliotecă folosită pentru citirea, scrierea, afișarea și procesarea imaginilor;
- **CMake** – sistem de build folosit pentru configurarea proiectului;
- **CLion** – mediu de dezvoltare folosit pentru rularea și testarea aplicației;
- **BMP grayscale 8 biți/pixel** – formatul principal al imaginilor de intrare;
- **AVI + MJPG** – formatul și codecul folosite pentru video-ul final.

Fișierul `CMakeLists.txt` configurează proiectul, setează standardul C++17, caută instalarea OpenCV și leagă bibliotecile necesare. De asemenea, definește macro-ul `PROJECT_ROOT`, folosit în cod pentru a construi căi absolute către directoarele proiectului.

2.3 Meniul Aplicației

Aplicația rulează în consolă și oferă un meniu interactiv. Opțiunile disponibile sunt:

```
1 ===== Project_PI - 3D Anaglyph Video =====
2 1 - Test OpenCV
3 2 - Convert raw datasets to BMP grayscale
4 3 - Generate simple anaglyph
5 4 - Generate disparity/depth map
6 5 - Generate anaglyph using depth map
7 6 - Process full stereo sequence
8 7 - Generate final video
9 0 - Exit
```

Fiecare opțiune apelează o funcție diferită din aplicație:

- opțiunea 1 verifică dacă OpenCV este configurat corect;
- opțiunea 2 convertește dataset-uri brute în imagini BMP grayscale;

-
- opțiunea 3 generează un anaglif simplu pentru prima pereche stereo;
 - opțiunea 4 generează harta de disparitate pentru prima pereche stereo;
 - opțiunea 5 generează un anaglif folosind depth map-ul calculat;
 - opțiunea 6 procesează întreaga secvență stereo;
 - opțiunea 7 generează video-ul final din cadrele anaglife.

Capitolul 3

Arhitectura Sistemului și Pipeline-ul de Procesare

3.1 Descriere Generală

Pentru a păstra proiectul clar și ușor de extins, aplicația este împărțită în mai multe module. Fiecare modul are o responsabilitate bine definită. Astfel, logica de citire a imaginilor este separată de logica de calcul a disparității, de generarea imaginilor anaglife și de construirea fișierului video.

Această organizare reduce complexitatea codului și permite testarea separată a fiecărei componente.

3.2 Diagrama Pipeline-ului

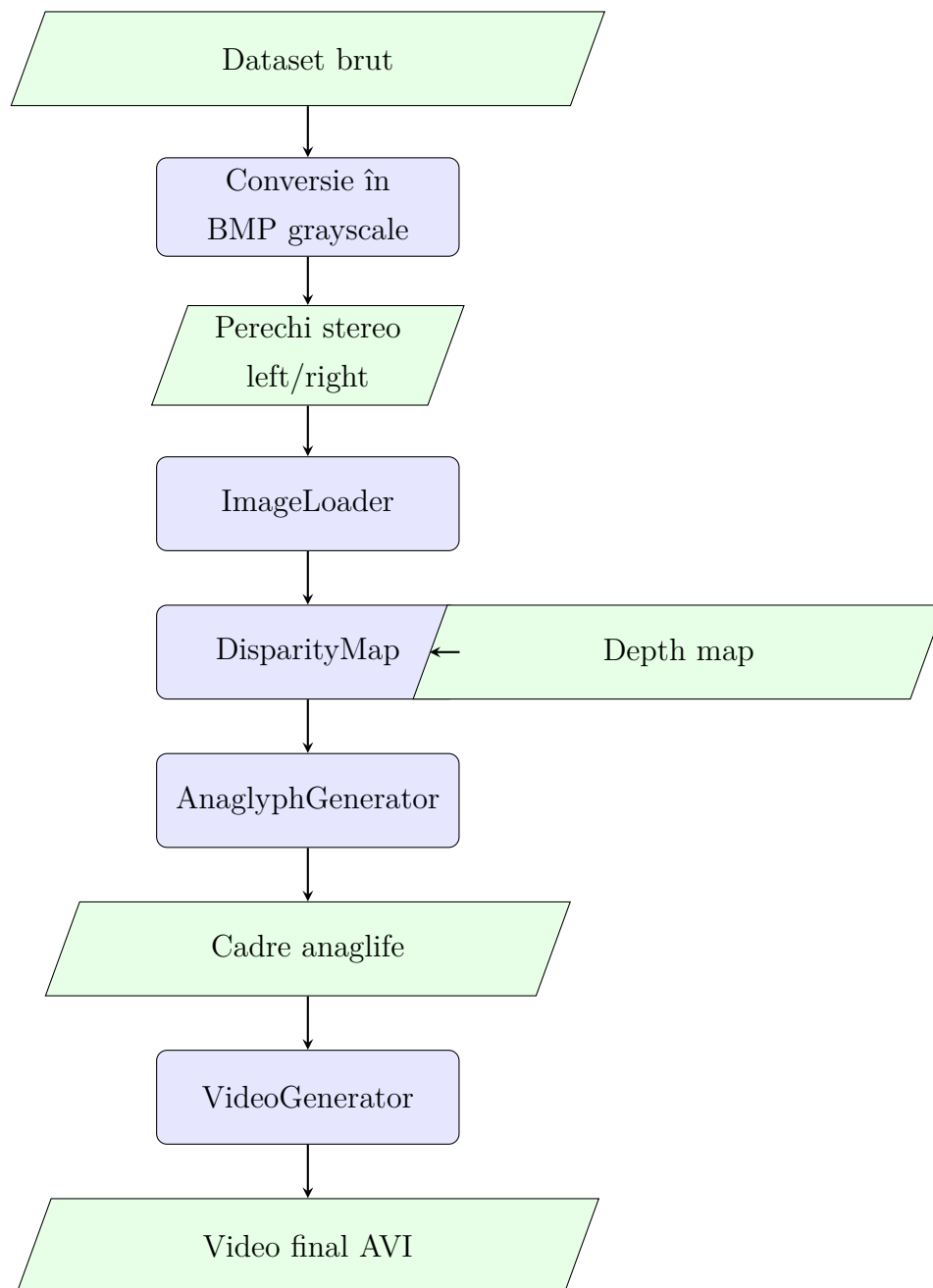


Figura 3.1: Pipeline-ul general al aplicației.

3.3 Module Principale

3.3.1 ImageLoader

Clasa `ImageLoader` se ocupă de operațiile de bază legate de fișiere și imagini. Aceasta verifică existența fișierelor, încarcă imagini grayscale, validează tipul imaginilor și construiește căile pentru cadre numerotate.

Funcțiile importante sunt:

- `fileExists` – verifică dacă un fișier există;
- `loadGrayscaleImage` – încarcă o imagine și verifică dacă este de tip `CV_8UC1`;
- `loadStereoPair` – încarcă o pereche stereo și verifică dacă imaginile au aceeași dimensiune;
- `framePath` – construiește calea unui fișier de forma `prefix_000.bmp`;
- `findAvailableFrameIndices` – caută perechile stereo disponibile.

3.3.2 DisparityMap

Clasa `DisparityMap` conține partea matematică principală a proiectului. Aceasta calculează harta de disparitate folosind algoritmul SAD, apoi normalizează rezultatul în intervalul 0–255 pentru a putea fi salvat ca imagine grayscale.

Cele mai importante funcții sunt:

- `computeDisparityMap` – calculează disparitatea brută;
- `normalizeDisparityMap` – normalizează valorile disparității;
- `generateDisparityMapFromFiles` – generează depth map pentru prima pereche stereo;
- `computeSAD` – variantă separată pentru calculul SAD;
- `normalizeToGrayscale` – normalizează o hartă brută în funcție de disparitatea maximă.

3.3.3 AnaglyphGenerator

Clasa `AnaglyphGenerator` generează imaginile anaglife. Există două variante:

- `createSimpleAnaglyph` – combină direct imaginea stângă și imaginea dreaptă;
- `createAnaglyphWithDisparity` – folosește depth map-ul pentru a deplasa pixelii și a accentua efectul 3D.

În OpenCV, ordinea canalelor este BGR, nu RGB. De aceea, canalul roșu este `color[2]`, canalul verde este `color[1]`, iar canalul albastru este `color[0]`.

3.3.4 VideoGenerator

Clasa `VideoGenerator` construiește video-ul final din cadrele anaglife generate anterior. Aceasta citește cadrele din `data/output/frames`, le verifică dimensiunea și le scrie într-un fișier video folosind `cv::VideoWriter`.

Pentru video se folosește codecul MJPG:

```
1 VideoWriter::fourcc('M', 'J', 'P', 'G')
```

Capitolul 4

Modele Matematice și Implementare

4.1 Calculul Hărții de Disparitate prin SAD

Cea mai importantă parte a proiectului este calculul hărții de disparitate. Pentru fiecare pixel din imaginea stângă se caută un pixel corespondent în imaginea dreaptă. Deoarece imaginile sunt considerate aliniate, căutarea se face doar pe orizontală.

Pentru fiecare valoare posibilă a disparității d , se compară o fereastră din imaginea stângă cu o fereastră deplasată din imaginea dreaptă. Metrica folosită este **SAD**, adică *Sum of Absolute Differences* [1].

Formula folosită este:

$$SAD(x, y, d) = \sum_{j=-R}^R \sum_{i=-R}^R |I_L(x+i, y+j) - I_R(x+i-d, y+j)| \quad (4.1)$$

unde:

- I_L este imaginea stângă;
- I_R este imaginea dreaptă;
- W este dimensiunea ferestrei;
- $R = \lfloor W/2 \rfloor$ este raza ferestrei;
- d este disparitatea testată.

Pentru fiecare pixel se reține disparitatea cu cel mai mic cost:

$$d^*(x, y) = \arg \min_d SAD(x, y, d) \quad (4.2)$$

În implementare, valorile folosite sunt:

-
- `maxDisparity = 64;`
 - `windowSize = 7.`

4.2 Normalizarea Depth Map-ului

Rezultatul brut al disparității conține valori într-un interval care depinde de imagine și de parametrul `maxDisparity`. Pentru a putea vizualiza rezultatul ca imagine grayscale, valorile sunt scalate în intervalul 0–255.

Formula generală de normalizare este:

$$P_{norm} = \frac{P - P_{min}}{P_{max} - P_{min}} \cdot 255 \quad (4.3)$$

Dacă valoarea maximă este egală cu valoarea minimă, imaginea normalizată rămâne neagră, deoarece nu există variație de disparitate.

4.3 Generarea Anaglifului Simplu

Pentru generarea anaglifului simplu, se parcurg pixelii celor două imagini grayscale. Imaginea finală este o matrice cu trei canale de tip `CV_8UC3`. Pentru fiecare pixel:

$$B(x, y) = I_R(x, y)$$

$$G(x, y) = I_R(x, y)$$

$$R(x, y) = I_L(x, y)$$

Astfel, imaginea stângă este introdusă în canalul roșu, iar imaginea dreaptă în canalele verde și albastru. Rezultatul este o imagine anaglifă roșu-cyan [2].

4.4 Generarea Anaglifului cu Depth Map

În varianta cu depth map, harta de disparitate este folosită pentru a calcula o deplasare orizontală. Valoarea depth map-ului este normalizată între 0 și 255, iar în cod este transformată într-o deplasare maximă de 16 pixeli:

$$shift = \frac{depthValue \cdot maxShift}{255} \quad (4.4)$$

unde `maxShift = 16`. Pixelul folosit pentru canalele verde și albastru este preluat dintr-o poziție deplasată:

$$shiftedX = x - shift \tag{4.5}$$

Această metodă accentuează efectul de paralaxă și face ca imaginea finală să aibă un efect 3D mai vizibil.

Capitolul 5

Fluxul de Rulare al Aplicației

5.1 Pregătirea Datelor

Pentru rularea aplicației, imaginile trebuie să fie organizate în două foldere:

```
1 data/left/left_000.bmp
2 data/left/left_001.bmp
3 data/left/left_002.bmp
4
5 data/right/right_000.bmp
6 data/right/right_001.bmp
7 data/right/right_002.bmp
```

Fiecare imagine din stânga trebuie să aibă perechea corespunzătoare în dreapta. De exemplu, `left_000.bmp` trebuie să corespundă cu `right_000.bmp`.

5.2 Conversia Dataset-ului Brut

Opțiunea 2 din meniu convertește imagini brute din `data/raw/boats` în imagini BMP grayscale. Funcția caută mai multe formate posibile, precum:

- `.ppm`;
- `.jpg`;
- `.jpeg`;
- `.png`;
- `.bmp`.

Pentru dataset-uri de tip multi-view, aplicația folosește un *baseline* de 2 cadre. Astfel, o imagine `imX` este asociată cu imaginea `imX+2`. Pentru secvențele de tip `image0_X` și `image1_X`, imaginile sunt asociate direct după index.

5.3 Procesarea unei Singure Perechi Stereo

Pentru prima pereche stereo, se pot rula următoarele opțiuni:

- opțiunea 3 generează `anaglyph_000.bmp`;
- opțiunea 4 generează `depth_000.bmp`;
- opțiunea 5 generează `anaglyph_depth_000.bmp`.

Această etapă este utilă pentru testarea rapidă a algoritmilor pe o singură pereche de imagini.

5.4 Procesarea Secvenței Complete

Opțiunea 6 parcurge toate perechile stereo numerotate consecutiv de la 0 la 999. Pentru fiecare index, aplicația verifică existența fișierelor:

```
1 data/left/left_XXX.bmp
2 data/right/right_XXX.bmp
```

Dacă una dintre imagini lipsește, procesarea se oprește. Pentru fiecare pereche validă se generează:

- un depth map în `data/output/depth_maps`;
- un cadru anaglif în `data/output/frames`.

5.5 Generarea Video-ului Final

Opțiunea 7 citește cadrele anaglife generate anterior și le combină într-un fișier video:

```
1 data/output/anaglyph_video.avi
```

Frame rate-ul folosit în implementare este:

```
1 int fps = 2;
```

Acest FPS redus este potrivit pentru vizualizarea clară a cadrelor generate și pentru observarea efectului 3D.

Capitolul 6

Analiza Performanței și Rezultate

6.1 Dataset-uri Utilizate

Sistemul a fost testat pe secvențe stereo formate din perechi de imagini. În proiect au fost folosite dataset-uri de tip *Cones* și *Boats*.

- **Cones** este un dataset potrivit pentru testarea disparității pe obiecte apropiate și bine texturate.
- **Boats** este o secvență mai mare, utilă pentru testarea procesării batch și pentru generarea unui video mai lung.

În cazul dataset-ului *Boats*, aplicația poate genera o secvență mai lungă de cadre, ceea ce demonstrează că pipeline-ul funcționează pentru mai multe perechi stereo consecutive.

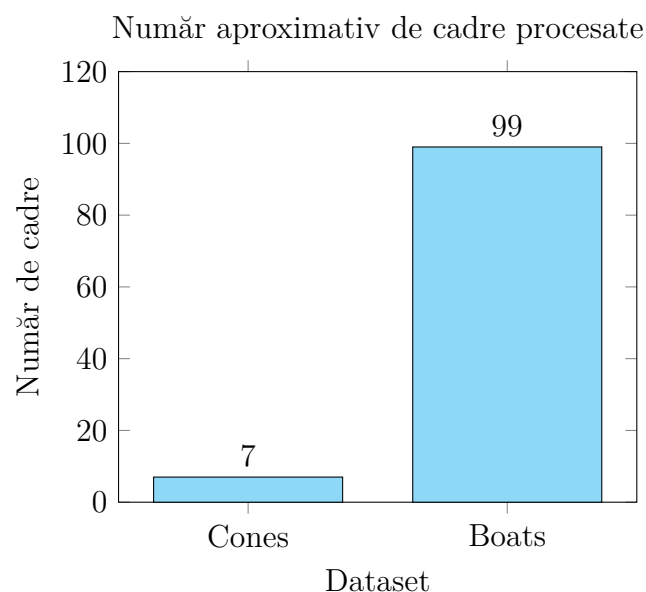


Figura 6.1: Exemplu de volum de cadre procesate pentru dataset-urile folosite.

6.2 Analiza Complexității

Algoritmul SAD folosit este o metodă de tip forță brută. Pentru fiecare pixel, se testează mai multe valori posibile ale disparității, iar pentru fiecare valoare se compară două ferestre de dimensiune $W \times W$.

Complexitatea algoritmului este:

$$O(N \cdot D \cdot W^2) \tag{6.1}$$

unde:

- N este numărul total de pixeli;
- D este disparitatea maximă;
- W este dimensiunea ferestrei.

Această complexitate explică de ce procesarea poate deveni lentă pentru imagini mari sau pentru valori mari ale disparității maxime. Totuși, pentru scop educațional, metoda este potrivită deoarece este clară, ușor de implementat și ușor de explicat.

6.3 Rezultate Obținute

Rezultatele principale ale aplicației sunt:

- hărți de disparitate salvate ca imagini grayscale;
- cadre anaglife roșu-cyan;
- video final în format AVI.

Hărțile de disparitate evidențiază diferențele dintre imaginea stângă și imaginea dreaptă. Zonele în care corespondentul este găsit la o deplasare mai mare apar mai luminoase după normalizare, iar zonele cu deplasare mică apar mai întunecate.

Cadrele anaglife combină informația celor două perspective într-o singură imagine color. Efectul 3D devine vizibil atunci când imaginea sau video-ul sunt privite cu ochelari roșu-cyan.

6.4 Limitări

Implementarea are câteva limitări importante:

-
- căutarea corespondentului se face doar pe orizontală;
 - algoritmul presupune că imaginile sunt alinierte;
 - SAD poate da rezultate slabe în zone fără textură;
 - marginile imaginii nu primesc întotdeauna valori de disparitate relevante;
 - nu se face calibrare stereo reală;
 - nu se folosesc optimizări GPU sau paralelizare.

Aceste limitări sunt normale pentru o implementare educațională și pot fi abordate în versiuni viitoare.

Capitolul 7

Concluzii, Contribuții și Dezvoltări Ulterioare

7.1 Concluzii

Proiectul demonstrează modul în care pot fi folosite conceptele de procesare stereo pentru a genera imagini și secvențe video anaglife. Aplicația pornește de la perechi de imagini grayscale, calculează hărți de disparitate folosind SAD și generează cadre roșu-cyan care pot fi combinate într-un video final.

Din punct de vedere tehnic, proiectul combină mai multe noțiuni importante din procesarea imaginilor:

- citirea și validarea imaginilor;
- lucrul cu matrice OpenCV;
- compararea ferestrelor de pixeli;
- generarea hărților grayscale;
- manipularea canalelor de culoare BGR;
- scrierea cadrelor într-un fișier video.

Proiectul este organizat modular, ceea ce îl face ușor de înțeles și de extins. Fiecare clasă are o responsabilitate clară, iar fluxul de procesare este împărțit în etape logice.

7.2 Contribuțiile Membrilor Echipei

Abordarea acestui proiect a implicat o împărțire colaborativă a responsabilităților.

-
- **Ghineț Ioana-Teodora:** s-a ocupat de partea de arhitectură modulară, încărcare a imaginilor, organizare a datelor de intrare și generare a video-ului final. A contribuit la implementarea claselor `ImageLoader`, `AnaglyphGenerator` și `VideoGenerator`, precum și la integrarea acestora în meniul aplicației.
 - **Murariu Georgiana:** s-a ocupat de partea matematică a proiectului, în special de calculul hărții de disparitate folosind SAD, normalizarea rezultatului și explicarea formulelor teoretice. A contribuit și la documentarea pipeline-ului, a arhitecturii și a limitărilor implementării.

7.3 Dezvoltări Ulterioare

Proiectul poate fi extins în mai multe direcții:

1. **Optimizarea algoritmului SAD:** se pot folosi imagini integrale sau alte metode pentru reducerea timpului de calcul.
2. **Filtrarea depth map-ului:** se poate aplica un filtru median pentru reducerea zgomotului din harta de disparitate.
3. **Folosirea unor algoritmi stereo avansați:** se pot compara rezultatele cu metode precum `StereoBM` sau `StereoSGBM`.
4. **Parametri configurabili:** valorile `maxDisparity`, `windowSize`, `fps` și `maxShift` pot fi citite de la tastatură sau dintr-un fișier de configurare.
5. **Interfață grafică:** aplicația poate fi extinsă cu o interfață GUI pentru alegerea imaginilor și vizualizarea rezultatelor.
6. **Export video MP4:** dacă sistemul are codecurile necesare, video-ul poate fi exportat și în format MP4.
7. **Calibrare stereo reală:** pentru rezultate mai precise, se poate adăuga calibrarea camerelor și rectificarea imaginilor.

Referințe Bibliografice

- [1] Daniel Scharstein și Richard Szeliski, “A taxonomy and evaluation of dense two-frame stereo correspondence algorithms”, în *International journal of computer vision* 47.1 (2002), pp. 7–42.
- [2] Andrew J Woods, “Understanding Crosstalk in Stereoscopic Displays”, în *Keynote presentation at 3DSA, Taiwan*, 2012.

Anexa A

Anexe: Snippet-uri de Cod Esențiale

A.1 Calculul Hărții de Disparitate

```
1 Mat DisparityMap::computeDisparityMap(const Mat& left,
2                                     const Mat& right,
3                                     int maxDisparity,
4                                     int windowSize)
5 {
6     if (left.empty() || right.empty())
7     {
8         cout << "Eroare: imaginile pentru disparity map sunt goale." <<
endl;
9         return Mat();
10    }
11
12    if (left.size() != right.size())
13    {
14        cout << "Eroare: imaginile pentru disparity map nu au aceeasi
dimensiune." << endl;
15        return Mat();
16    }
17
18    if (left.type() != CV_8UC1 || right.type() != CV_8UC1)
19    {
20        cout << "Eroare: imaginile trebuie sa fie grayscale pe 8 biti/pixel
." << endl;
21        return Mat();
22    }
23
24    if (windowSize < 3)
25    {
26        windowSize = 3;
```

```

27     }
28
29     if (windowSize % 2 == 0)
30     {
31         windowSize++;
32     }
33
34     if (maxDisparity < 1)
35     {
36         maxDisparity = 1;
37     }
38
39     if (maxDisparity > 255)
40     {
41         maxDisparity = 255;
42     }
43
44     int radius = windowSize / 2;
45     Mat disparity(left.rows, left.cols, CV_8UC1, Scalar(0));
46
47     for (int y = radius; y < left.rows - radius; y++)
48     {
49         for (int x = radius; x < left.cols - radius; x++)
50         {
51             int bestDisparity = 0;
52             int bestCost = INT_MAX;
53
54             for (int d = 0; d <= maxDisparity; d++)
55             {
56                 if (x - d - radius < 0)
57                 {
58                     continue;
59                 }
60
61                 int cost = 0;
62
63                 for (int wy = -radius; wy <= radius; wy++)
64                 {
65                     for (int wx = -radius; wx <= radius; wx++)
66                     {
67                         int leftValue = left.at<uchar>(y + wy, x + wx);
68                         int rightValue = right.at<uchar>(y + wy, x + wx - d
69
70 );
71
72                         cost += abs(leftValue - rightValue);
73                     }
74                 }
75             }
76         }
77     }

```

```

71         }
72
73         if (cost < bestCost)
74         {
75             bestCost = cost;
76             bestDisparity = d;
77         }
78     }
79
80     disparity.at<uchar>(y, x) = static_cast<uchar>(bestDisparity);
81 }
82 }
83
84 return disparity;
85 }

```

A.2 Normalizarea Hărții de Disparitate

```

1 Mat DisparityMap::normalizeDisparityMap(const Mat& disparity)
2 {
3     if (disparity.empty())
4     {
5         cout << "Eroare: disparity map-ul este gol si nu poate fi
6         normalizat." << endl;
7         return Mat();
8     }
9
10    double minValue;
11    double maxValue;
12    minMaxLoc(disparity, &minValue, &maxValue);
13
14    Mat normalized(disparity.rows, disparity.cols, CV_8UC1, Scalar(0));
15
16    if (maxValue <= minValue)
17    {
18        return normalized;
19    }
20
21    for (int y = 0; y < disparity.rows; y++)
22    {
23        for (int x = 0; x < disparity.cols; x++)
24        {
25            int value = disparity.at<uchar>(y, x);

```

```

25         int normalizedValue =
26             static_cast<int>((value - minValue) * 255.0 / (maxValue -
normalizedValue));
27
28         normalized.at<uchar>(y, x) = static_cast<uchar>(normalizedValue
);
29     }
30 }
31
32 return normalized;
33 }

```

A.3 Generarea Anaglifului Simplu

```

1 Mat AnaglyphGenerator::createSimpleAnaglyph(const Mat& leftImage,
2                                             const Mat& rightImage)
3 {
4     Mat anaglyph(leftImage.rows, leftImage.cols, CV_8UC3);
5
6     for (int y = 0; y < leftImage.rows; y++)
7     {
8         for (int x = 0; x < leftImage.cols; x++)
9         {
10             uchar leftPixel = leftImage.at<uchar>(y, x);
11             uchar rightPixel = rightImage.at<uchar>(y, x);
12
13             Vec3b color;
14             color[0] = rightPixel; // B
15             color[1] = rightPixel; // G
16             color[2] = leftPixel;  // R
17
18             anaglyph.at<Vec3b>(y, x) = color;
19         }
20     }
21
22     return anaglyph;
23 }

```

A.4 Generarea Anaglifului cu Depth Map

```

1 Mat AnaglyphGenerator::createAnaglyphWithDisparity(const Mat& leftImage,
2                                                     const Mat& rightImage,

```

```

3                                     const Mat& disparityMap)
4 {
5     Mat anaglyph(leftImage.rows, leftImage.cols, CV_8UC3);
6     int maxShift = 16;
7
8     for (int y = 0; y < leftImage.rows; y++)
9     {
10         for (int x = 0; x < leftImage.cols; x++)
11         {
12             uchar leftPixel = leftImage.at<uchar>(y, x);
13             uchar depthValue = disparityMap.at<uchar>(y, x);
14
15             int shift = depthValue * maxShift / 255;
16             int shiftedX = x - shift;
17
18             uchar rightPixel = 0;
19             if (shiftedX >= 0 && shiftedX < leftImage.cols)
20             {
21                 rightPixel = leftImage.at<uchar>(y, shiftedX);
22             }
23
24             Vec3b color;
25             color[0] = rightPixel; // B
26             color[1] = rightPixel; // G
27             color[2] = leftPixel;  // R
28
29             anaglyph.at<Vec3b>(y, x) = color;
30         }
31     }
32
33     return anaglyph;
34 }

```

A.5 Încărcarea unei Perechi Stereo

```

1 bool ImageLoader::loadStereoPair(const string& leftPath,
2                                   const string& rightPath,
3                                   Mat& leftImage,
4                                   Mat& rightImage)
5 {
6     if (!loadGrayscaleImage(leftPath, leftImage))
7     {
8         return false;
9     }

```

```

9     }
10
11     if (!loadGrayscaleImage(rightPath, rightImage))
12     {
13         return false;
14     }
15
16     if (leftImage.size() != rightImage.size())
17     {
18         cout << "Imaginile stereo nu au aceeasi dimensiune." << endl;
19         cout << "Left: " << leftImage.cols << "x" << leftImage.rows << endl
20         ;
21         cout << "Right: " << rightImage.cols << "x" << rightImage.rows <<
22         endl;
23         return false;
24     }
25
26     return true;
27 }

```

A.6 Generarea Video-ului Final

```

1 bool VideoGenerator::createVideoFromFrames(const string& framesFolder,
2                                             const string& outputPath,
3                                             int fps)
4 {
5     vector<Mat> frames;
6
7     for (int index = 0; index <= 999; index++)
8     {
9         string framePath = ImageLoader::framePath(framesFolder, "anaglyph",
10         index);
11
12         if (!ImageLoader::fileExists(framePath))
13         {
14             if (index == 0)
15             {
16                 cout << "Nu exista primul frame: " << framePath << endl;
17             }
18             break;
19         }
20
21         Mat frame = imread(framePath, IMREAD_COLOR);

```

```

21     if (frame.empty())
22     {
23         cout << "Nu pot citi frame-ul: " << framePath << endl;
24         continue;
25     }
26
27     frames.push_back(frame);
28 }
29
30 if (frames.empty())
31 {
32     cout << "Nu exista cadre anaglife cu depth map in: " <<
framesFolder << endl;
33     return false;
34 }
35
36 Size frameSize(frames[0].cols, frames[0].rows);
37
38 VideoWriter writer(outputPath,
39                     VideoWriter::fourcc('M', 'J', 'P', 'G'),
40                     fps,
41                     frameSize,
42                     true);
43
44 if (!writer.isOpened())
45 {
46     cout << "Nu pot crea video-ul: " << outputPath << endl;
47     return false;
48 }
49
50 for (const Mat& frame : frames)
51 {
52     if (frame.size() != frameSize)
53     {
54         cout << "Frame ignorat deoarece are alta dimensiune." << endl;
55         continue;
56     }
57
58     writer.write(frame);
59 }
60
61 writer.release();
62
63 cout << "Video generat: " << outputPath << endl;
64 cout << "Numar cadre: " << frames.size() << ", FPS: " << fps << endl;

```

```
65
66     return true;
67 }
```

A.7 Meniul Principal al Aplicației

```
1  int main()
2  {
3      int option;
4
5      do
6      {
7          cout << "\n==== Project_PI - 3D Anaglyph Video =====\n";
8          cout << "1 - Test OpenCV\n";
9          cout << "2 - Convert raw datasets to BMP grayscale\n";
10         cout << "3 - Generate simple anaglyph\n";
11         cout << "4 - Generate disparity/depth map\n";
12         cout << "5 - Generate anaglyph using depth map\n";
13         cout << "6 - Process full stereo sequence\n";
14         cout << "7 - Generate final video\n";
15         cout << "0 - Exit\n";
16         cout << "Option: ";
17         cin >> option;
18
19         switch (option)
20         {
21             case 1:
22                 testOpenCV();
23                 break;
24
25             case 2:
26                 convertDatasetToBmp(projectPath(), "boats");
27                 break;
28
29             case 3:
30                 generateSimpleAnaglyphFromFiles();
31                 break;
32
33             case 4:
34                 DisparityMap::generateDisparityMapFromFiles(projectPath());
35                 break;
36
37             case 5:
```

```
38         generateAnaglyphUsingDepthMap();
39         break;
40
41     case 6:
42         processFullStereoSequence();
43         break;
44
45     case 7:
46         generateFinalVideo();
47         break;
48
49     case 0:
50         cout << "Exit.\n";
51         break;
52
53     default:
54         cout << "Invalid option.\n";
55         break;
56     }
57
58 } while (option != 0);
59
60 return 0;
61 }
```